

La sintaxis del lenguaje

Convenciones de nombres

Nombres descriptivos

- Una mala costumbre habitual cuando se empieza en la programación es darle un nombre muy poco descriptivo
- A medida que el código crece, se vuelve insostenible
- Evita nombres poco claros o inconsistentes como `tmp`, `a`, `b2`, `variable2`, etc

Índices y contadores

- Cuando trabajamos en bucles `for` (o bucles en general)
- El ámbito de una variable que actúa como contador (índice) es muy reducido (esa variable solo existe y afecta al interior del bucle)
- Las variables que actúan como contador suelen nombrarse con una letra minúscula empezando desde `i`: `i`, `j`, `k`, etc. A veces también se usan letras como `a`, `b`, `c` ... o la inicial minúscula de lo que representan: `c` para un contador, `p` para una posición, etc.

Constantes, clases y variables

- Las **constantas** son variables que no varían su valor a lo largo del programa. Deben ir siempre en **MAYÚSCULAS**
- Las **clases** son estructuras de código más complejas. Los nombres de las clases se escriben en `PascalCase`.
- Las variables, por último siempre en **camelCase**

Tipos de datos

- JS es un lenguaje de **tipado débil** porque cuando se declara una variable no se indica su tipo.
- JS es un lenguaje de **tipado dinámico** porque las variables pueden cambiar de tipo en tiempo de ejecución.
- Existen dos tipos de datos:
 - **Primitivos** (inmutables)
 - **Boolean**
 - **Number** (64 bits)
 - **String**
 - **Null**
 - **Undefined**
 - **Objetos** (mutables, todo lo que no sean primitivos, incluyendo funciones)

Ejemplos

```
let numero1=1;      console.log(typeof numero1);    //number
let numero2=1.03;   console.log(typeof numero2);    //number
let booleano = true; console.log(typeof booleano);   //boolean
let cadena="Hola";  console.log(typeof cadena);     //string
let nulo=null;      console.log(typeof nulo);        //object
                  console.log(nulo);                //null
let indefinido;     console.log(typeof indefinido);  //undefined
                  console.log(indefinido);           //undefined
let objeto = {a:1}; console.log(typeof objeto);     //object
```

El operador "typeof" devuelve el tipo de dato de una variable declarada.

```
let a=123;
a="Hola";
console.log(a); // Hola
```

El tipo de la variable puede cambiar en tiempo de ejecución sin problemas.

En algunos casos, typeof resulta insuficiente porque en tipos de datos más avanzados simplemente nos indica que son objetos.

Con `constructor.name` podemos obtener el tipo de constructor que se utiliza.

Ejemplos:

```
console.log(text.constructor.name) // String
console.log(number.constructor.name) // Number
console.log(boolean.constructor.name) // Boolean
console.log(notDefined.constructor.name) // ERROR, sólo funciona con
variables definidas
```

Declaración de variables

- Se pueden declarar variables de tres formas
 - **Directamente** (`a = 2`). Es la opción menos recomendada. A partir de ES5 se recomienda usar el modo estricto escribiendo en la primera línea del fichero "use strict"; con lo que se impide declaraciones directas.
 - **Utilizando var** : permite declarar una variable cuyo ámbito es la función donde se define o global (si se define fuera de una función)

- **Utilizando `let`** : es la opción recomendada. La definición tiene ámbito de bloque de código, de función o global si se define en la raíz del documento. Disponible a partir de ES6.
- Las constantes se definen con `const` y tienen ámbito de bloque.

Ámbito de las variables

```
let a1=1;           // Ámbito global
//...
{                   // Ámbito de bloque
    let a2=2;
    //...
}
function f() {      // Ámbito de función
    let a3=3;
    //...
    if (true){      // Ámbito de bloque
        let a4=4;
        //...
    }
}
```

Las funciones **IIFE** (Immediately-invoked function expressions) o **funciones autoinvocadas** permiten encapsular variables declaradas mediante `var` creando un ámbito local

```
(function() {
    //....
})();
```

```
(function(n) {
    var saludo="Hola"
    console.log(saludo+n) // Hola Juan
})("Juan")
```

Ámbito de las variables. Hoisting

```
console.log(a); //undefined
var a=1;
{
  console.log(a); // 1
}

(function() {
  console.log(a); //1
  console.log(b); //undefined
  var b=2;
  console.log(b); //2
})();
console.log(b); //Error: b is not defined
```

```
console.log(a); //Error: a is not defined
let a=1;
{
  console.log(a); // 1
}

(function() {
  console.log(a); // 1
  console.log(b); //Error: b is not defined
  let b=2;
  console.log(b); // 2
})();
console.log(b); //Error: b is not defined
```

Ámbito de las variables. Declaración con var

```
var a=1;
console.log(1); // 1
{
  console.log(a); // 1
}
(function f() {
  console.log(a); // 1
  if (true) {
    console.log(a); // 1
  }
})
```

```

var a=1;
console.log(1); // 1
(function f(){
    var a=2;
    console.log(a) // 2
    if (true) {
        var a=3;
        console.log(a); // 3
    }
    console.log(a); // 3
})();
console.log(a); // 1
var a=5;
console.log(a); // 5;

```

Ámbito de las variables. Declaración con let

```

let a=1;
console.log(1); // 1
{
    console.log(a); // 1
}
(function f(){
    console.log(a); // 1
    if (true){
        console.log(a); // 1
    }
    console.log(a); //1
})();

```

```

let a = 1;
console.log(1); // 1
(function f(){
    let a = 2;
    console.log(a); // 2
    if (true){
        let a=3;
        console.log(a); // 3
    }
    console.log(a); // 2
})();
console.log(a); // 1
let a = 5; // Error: already declared

```

Operadores

Aritméticos

- Suma: `a+b`
- Resta: `a-b`
- Multiplicación: `a*b`
- División: `a/b`
- Resto de división: `a%b`
- Exponenciación: `a**b`
- Incremento: `++`
- Decremento: `--`

Ternario

```
(condicion)? instruccion1: instruccion2
```

Concatenación de strings

```
"string1" + "string2"
```

Operadores lógicos

- AND: `a&b`
- OR: `a||b`
- NOT: `!a`

Bitwise

- AND: `a&b`
- OR: `a|b`
- XOR: `a^b`
- NOT: `~a`
- Shift Left: `a<<b`
- Shift Right: `a>>b`

Operaciones de comparación

- `>, <, >=, <=, ==, !=`
- `===` (igualdad estricta)
- `!==` (desigualdad estricta)

Propiedades globales y funciones globales:

- `Infinity` : Representa el valor infinito
- `NaN` : Not a number
- `null` : valor nulo
- `isFinite()` , `isNaN()` , `parseFloat()` , `parseInt()`

Conversión de tipos

- Se puede cambiar el tipo de las variables dinámicamente (incluso en el modo "use strict").
- Para convertir entre tipos se pueden utilizar los métodos:
 - `parseInt()`
 - `parseFloat()`
 - `toString()`
 - `eval()`

Cuando hacemos una asignación o como resultado de una expresión hay que tener en cuenta la `coerción de tipos`.

```
"use strict"
let a=1;
a="hola"
console.log(a); // hola
let b = parseInt("123.45");
console.log(b) // 123
let c = parseFloat("123.45");
console.log(c) // 123.45
let d = 46;
console.log(d.toString(2)); // 101110
console.log(d.toString(16)); // 2e
let e=eval("2+4+parseInt(1.2)");
console.log(e); // 7
```